Chapter 12 exercises • File-system implementation

Build state and variable tables before consulting the separate answers.

OS12-E01 • Indexed allocation

Block size is 4 KiB, pointers four bytes, and the inode has twelve direct pointers.

1. How much data does single indirection address?
2. How many data-pointer entries does double indirection address?
3. What metadata space do three indirect blocks consume?

OS12-E02 • Crash consistency

First consider an unjournaled create whose inode reached durable storage but directory entry did not before power failure. For part 2, instead use a redo-only, no-steal metadata journal: complete after-images persist before the durable commit record, uncommitted home metadata cannot be written, and log clearing follows a durable checkpoint. Data durability has its own contract.

1. What can recovery do if only the inode write completed?
2. How can journaling handle the transition?
3. Does fsck guarantee that data previously read successfully remains after failure?

OS12-E03 • Free-block structures

Use 1=free,0=allocated over 1024 blocks; blocks 7,8,9 are allocated.

1. What data is logically needed to test whether block 8 is free?
2. What other cost can a free list incur?
3. How can duplicate allocation be prevented?

OS12-E04 • Path walk, open tables, and reads

Blocks are 4 KiB. Initially only root-directory B4 is cached. /home/u/notes.txt uses directories B4→B5→B6; inodes 3/5/7 share inode-table B3. File data is blocks 8,9,10, size 12 KiB. P1 opens this regular file with read permission; its read(8192) succeeds and actually returns 8192 B. P2 then independently opens the same file with read permission. Each directory is searchable; B8/B9 are initially uncached. Count only the stated metadata and data block reads. A successful read can generally be short; the return length is fixed here.

1. Trace directory lookups, metadata reads, and inode/block results.
2. Which data blocks are read, and what are P1/P2 offsets?
3. Distinguish fd table, OFD, and inode/FCB contents. How many descriptions exist?
4. Does P2 lseek(4096,0) change P1's next read position?

OS12-E05 • Bitmap space and file extents

A 16-block free bitmap uses1=free,0=allocated and starts at 0000000100011111, with characters left to right representing blocks 0–15. First-fit searches upward from block 0. Inode7 owns8–10, size 12 KiB. Append12 KiB in three consecutive blocks.

1. List free blocks and the first-fit extent.
2. Give the updated bitmap/free set.
3. What can a crash between writing new inode pointers and the bitmap expose?
4. Compare I/O for the original file’ s logical block 2 (block 10), not the newly appended extent. Contiguous blocks are 8–10; embedded links start at head 8 and follow 8→9→10, with uncached pointers. The indexed entry points to block 10: compare cached versus cold index. Exclude inode and other caches; count only data/index block reads needed for this access.

OS12-E06 • Directory indexes and crash recovery

Hash names by sum(character codes) mod8. Insert alpha, beta, gamma, delta, epsilon, zeta, eta in order, appending within buckets. T7 starts at home M0, size 12288 with data blocks 8–10 allocated; M1 has size 24576 with blocks 8–13 allocated and a matching bitmap. New blocks 11–13 are reserved against concurrent reuse. T7 uses redo-only/no-steal ordered metadata journaling: data and complete metadata after-images persist before commit, then a durable checkpoint precedes log clearing.

1. Give buckets, chain lengths, and actual mean successful probes.
2. Explain capacity and correctness risks during resize.
3. Recover crashes after descriptor but before commit, after commit but before full home updates, and before clearing an already applied transaction.
4. Does committed metadata alone guarantee durable user data? State this model's ordering.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

OS12-E07 • Indirection and sparse boundaries

Use4 KiB blocks, four-byte pointers, twelve direct plus single/double/triple indirection. Inode is resident; unless stated cached, all index/data blocks are cold and each index level is a distinct block. The hole example lies within file size; a read starting at or after EOF returns 0 bytes.

1. Decompose offset 5 MiB+123 into logical block, byte displacement, and indirect indices.
2. Give first/last single/double logical blocks and this access's I/O count.
3. What does a sufficiently large sparse file return at that hole? Distinguish logical size, index space, and allocated data.

OS12-E08 • Open, dup, and EOF states

Size12288. P1 fd4 opens O1, P2 fd9 independently opens O2, both at 0. Execute P1 read(fd4,8192), dup to fd6, P1 read(fd6,4096), P2 lseek(fd9,4096, SEEK_SET), P1 read(fd4,1). There are no errors/other writers. The first two reads are specified to return exactly 8192 and 4096 bytes. A successful read can generally be short; this is an extra condition for this trace. The final read follows EOF semantics.

1. Give fd→description and both offsets after each step.
2. Explain the final result and why P2's seek does not alter it.
3. Is fd4 usable after closing fd6? How does fork inheritance differ?

OS12-E09 • Redo invariants and non-idempotent replay

T7 changes M0(size 12288, blocks 8–10) to M1(size 24576, blocks 8–13 with matching bitmap). Use ordered redo-only/no-steal, an atomically recognizable commit, and data/after-images durable first.

1. Prove recovery for precommit, postcommit bitmap-only home, and postcheckpoint uncleared-log crashes.
2. What does replaying size+=12288 and append11–13 twice do? Repair the record format.
3. Give a counterexample if uncommitted inode changes may reach home before discarding the log.
4. Are Sunday full B0 plus Tuesday incremental B2 sufficient after Wednesday failure when Monday incremental B1 exists? Can a snapshot replace an independent backup?

References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 12.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study